```
  ...
}
```

With this constructor, each `ProducerRecord` is converted into a `Message` and thus into an `Envelope`. The envelope has a `passThrough`, but in `plainSink` this is not used, as you've seen in `.map(Message(_, NotUsed))`.

With this in mind, let's return to `committableSink` and look at its signature:

```
package akka.kafka.scaladsl

object Producer {
  ...

  def committableSink[K, V](
      producerSettings: ProducerSettings[K, V],
      committerSettings: CommitterSettings
  ): Sink[Envelope[K, V, ConsumerMessage.Committable], Future[Done]]
  ...
}
```

It needs as input the producer settings, which specify, among other things, the serialization and the location of the bootstrap servers, just as with the `plainSink`.

However, the most revealing part is the signature of the output—more precisely, the type of elements that the `Sink` must consume: `Envelope[K, V, ConsumerMessage.Committable]`. You are again dealing with an `Envelope`, as you saw in `plainSink`, but now you use the `passThrough`. Here the `passThrough` is a `ConsumerMessage.Committable` instead of `NotUsed` as in the `plainSink`.